

Практическая работа 3. Реализация алгоритмов ветвления. Условные операторы.

[Задание 1. Теория](#)

[Задание 2. Выполните задания базового уровня \(А\)](#)

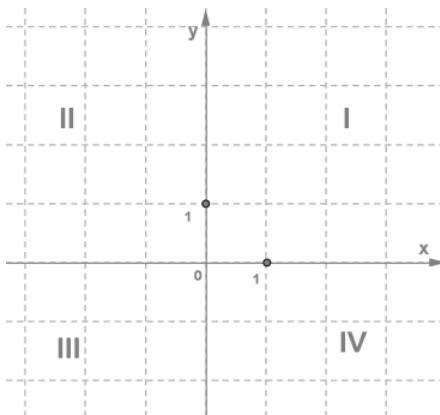
[Задание 3. Выполните задания продвинутого уровня \(В\)](#)

Задание 1. Теория

Ознакомьтесь с лекционным материалом на образовательном портале: [Лекция 9. Условные операторы.](#)

Задание 2. Выполните задания базового уровня (А)

А1. Вводятся координаты двух точек на плоскости: x_1 , y_1 , x_2 , y_2 . Все координаты не равны 0. Определить, лежат ли обе точки в одной координатной четверти. Если да, то определить в какой четверти лежат точки.



Ввод: 3 -3 5 -2 Вывод: Yes, IV	Ввод: 3 3 5 -1 Вывод: No
---	---

Для решения задачи потребуется записать логические выражения, включающие не только операции сравнения, но также некоторые логические операции: **and** (логическое умножение «И»), **or** (логическое сложение «ИЛИ»), **not** (логическое отрицание «НЕ»). Например,

```
if (x1>0 and y1>0):
    print("I четверть") # точка (x1,y1) лежит в первой координатной четверти
```

А2. Напишите программу, которая проверяет введенный пароль на надежность. Требования к надежному паролю следующие:

- Длина пароля – 8 символов
- В пароле должны быть заглавные и строчные буквы, цифры и специальные символы: *, - , #

В случае ввода надежного пароля выведите на печать «Надежный пароль». В остальных случаях выведите на печать все ошибки из следующих возможных, допущенные при вводе пароля:

- Длина пароля не равна 8;
- В пароле отсутствуют заглавные буквы;
- В пароле отсутствуют строчные буквы;
- В пароле отсутствуют цифры;
- В пароле отсутствуют специальные символы;
- В пароле используются непредусмотренные символы.

Ввод: qwerty11 Вывод: В пароле отсутствуют заглавные буквы В пароле отсутствуют специальные символы	Ввод: 8Yy#*rrT Вывод: Надежный пароль
Ввод: @#@@uU Вывод: Длина пароля не равна 8 В пароле отсутствуют цифры В пароле используются непредусмотренные символы	

Для решения задачи потребуется использовать условные операторы, а также некоторые возможности работы со строковыми значениями из модуля string. Импортируйте его:

```
import string
```

Длину введенного пароля (строки) можно получить, используя функцию len(). Функция вернет количество символов в строке.

```
len(password)
```

Функция lower() преобразует все символы строки в нижний регистр (строчные буквы):

```
password = '8Yy#*rrT'
print(password.lower()) # вывод - 8yy#*rrt
```

Сравнив исходный пароль с преобразованным к нижнему регистру паролем можно понять, есть ли в строке заглавные буквы. Аналогично, для строчных букв, но с использованием функции upper().

Определить, есть ли в исходной строке хотя бы одна цифра можно с использованием функций isdigit() и any(). Например, следующий ниже вызов функции any() вернет True, если в строке password есть хотя бы одна цифра:

```
any(symbol.isdigit() for symbol in password)
```

Запись `for symbol in password` означает, что каждый символ строки password будет проверен функцией isdigit() на то, является ли символом цифрой (тогда True) или нет (тогда False). Используйте конструкцию из примера в условном операторе if для проверки наличия цифр в строке.

Определить, есть ли в строке недопустимые символы можно следующим образом:

1. Склеим в одну строку все допустимые символы пароля (буквы в нижнем регистре + буквы в верхнем регистре + цифры + специальные символы *-#).

```
allowed = string.ascii_uppercase + string.ascii_lowercase + string.digits + '*-#'
```

Здесь string.ascii_uppercase (все буквы в верхнем регистре), string.ascii_lowercase (все буквы в нижнем регистре) и string.digits (все цифры) – это строковые константы модуля string. Подробнее по [ссылке](#). Для проверки выполните фрагмент кода:

```
allowed = string.ascii_uppercase + string.ascii_lowercase + string.digits + '*-#'  
print(allowed)
```

На экран будет выведена строка:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789*~#
```

2. Проверим, что все символы пароля присутствуют в этой строке. Если нет – пароль неверный. Это можно сделать, используя функцию all(). Функция возвращает True, если все элементы последовательности True. В нашем случае для каждого символа пароля password достаточно проверить, что он встречается в строке допустимых символов allowed. Если это так, то следующий вызов функции all() вернет True:

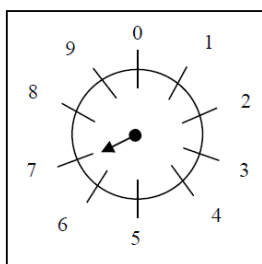
```
all(symbol in allowed for symbol in password)
```

Например, при выполнении следующего фрагмента кода на экран будет выведено True:

```
password = 'llqweQ-s'  
allowed = string.ascii_uppercase + string.ascii_lowercase + string.digits + '*-#'  
print(all(symbol in allowed for symbol in password))
```

А3. Газовая компания выставляет счет потребителям за каждый использованный кубический метр газа. Раз в месяц сотрудник компании считывает текущее показание счетчика. Компания использует показания за предыдущий месяц и за текущий месяц, чтобы определить, сколько газа потребил заказчик.

Счетчик газа представляет собой круглое устройство, на котором нанесены метки для чисел от 0 до 9999. Стрелка указывает на текущее показание счетчика и перемещается к следующей метке, когда использован один кубометр газа. Когда стрелка достигает отметки 9999 и используется еще один кубометр газа, то счетчик сбрасывается на 0. Ниже представлен рисунок, на котором нанесены метки только для каждой тысячи кубических метров. Стрелка указывает на 7 и это означает, что текущее показание счетчика – 7000 кубометров.



Сотрудник компании записывает текущее показание стрелки и использует показания предыдущего месяца для вычисления общего количества использованного газа. Так, на рисунке, если предыдущее показание было 4000, то объем использованного газа в текущем месяце $7000 - 4000 = 3000$ кубометров.

Компания вычисляет сумму денег, которую клиент должен заплатить за использованный газ:

- За любой объем до 300 кубометров (включительно) взимается фиксированная сумма – 21\$

- За любое количество более 300 кубометров, но меньше либо равное 600, дополнительно взимается 0,06\$ за каждый кубометр.
- За любой объем более 600 кубометров и менее или равный 800, компания берет 0,04\$ за кубический метр.
- Наконец, за любую сумму, превышающую 800 кубических метров, они взимают 0,025\$ за кубический метр.

Например, если общий объем использованного газа составляет 470 кубометров, то счет за газ будет рассчитан так:

$$21 + 170 * 0,06 = 31,20 \$$$

Здесь 21\$ покрывают первые 300 кубометров, а оставшиеся 170 оплачиваются из расчета 0,06\$ за каждый.

Напишите программу, которая по двум показаниям счетчика (предыдущее и текущее) определяет объем использованного газа в кубометрах, вычисляет сумму оплаты за газ и среднюю цену за кубометр. Сумму оплаты и средней цены за кубометр округлите при выводе до двух знаков после запятой.

Ввод: 575 1250 Вывод:				
Предыдущее	Текущее	Использовано	К оплате	Ср. цена m^3
575	1250	675	42.00	0.06

Ввод: 8725 150 Вывод:				
Предыдущее	Текущее	Использовано	К оплате	Ср. цена m^3
8725	150	1425	62.63	0.04

Ввод: 2620 4250 Вывод:				
Предыдущее	Текущее	Использовано	К оплате	Ср. цена m^3
2620	4250	1630	67.75	0.04

Задание 3. Выполните задания продвинутого уровня (В)

В1. Кот и мышь. Кот и мышь перемещаются в замкнутом пространстве в форме прямоугольной сетки, разделенной на клетки. Каждая клетка помечена парой координат (строка, столбец). На рисунке ниже мышь **М** находится в клетке (3, 7), а кот **С** находится в клетке (6, 2).

	1	2	3	4	5	6	7	8	9
1									
2									
3							М		
4									
5									
6		С							
7									
8									
9									

Каждый может перемещаться по сетке, но не может покидать сетку. Если кот и мышь одновременно достигают той же ячейки, то кот ловит мышь и игра заканчивается.

Каждый игрок (кот или мышь) перемещается по строкам и/или столбцам сетки. Ход указывается парой чисел. Например, ход (3, -1) определяет на сколько ячеек игрок должен переместиться по вертикали (первое число) и по горизонтали (второе число). Если бы кот сделал ход (3, -1) из своего местоположения, показанного на рисунке, то он бы оказался в клетке (9, 1).

Ни одному игроку не разрешается перемещаться за пределы сетки. Если сделать такой ход, игрок просто перемещается на противоположную сторону сетки. Например, если мышь попытается сделать ход (-4, 2) из своей начальной позиции, то окажется в клетке (8, 9). Если кот попытается сделать (5, -2) из своей исходной позиции, то окажется в клетке (2, 9).

Игроки перемещаются по рядам и столбцам, и соответственно рассчитываются пройденные ими расстояния. Учитывая начальные позиции, показанные на рисунке, расстояние между котом и мышью равно 8. Расстояния никогда не бывают отрицательными. Если кот сделала ход (3, -1) из своего начального положения (6, 2), то он прошел расстояние 4. Обратите внимание, что начальное положение не считается перемещением из (0, 0).

Напишите программу, которая считывает данные из файла ChaseData.txt, содержащего данные о начальных позициях и перемещениях игроков, определяет расстояние между игроками, расстояние, пройденное игроками за всю игру, а также исход игры – «Кот поймал мышь» или «Мышь сбежала от кота».

Формат входных данных:

28	24		
М		21	13
М		-11	9
Р			
М		-6	4
Р			
М		5	-9
С		27	8
М		4	-8
Р			
М		10	-7
Р			
С		12	-11
Р			
С		-3	9
М		7	6
Р			
М		9	2
С		-9	-10
Р			
М		-9	-2
Р			
С		-13	7
Р			

Первая строка – два целых числа (количество строк и столбцов сетки)

Каждая следующая строка представлена в одном из следующих форматов:

М <число> <число> // начальная позиция или ход мыши

С <число> <число> // начальная позиция или ход кота

Р // команда вывода на экран (print) текущего состояния игры

Формат выходных данных:

Cat and Mouse		
Cat	Mouse	Distance
(?, ?)	(10,22)	
(?, ?)	(4, 2)	
(27, 8)	(13, 9)	15
(27, 8)	(23, 2)	10
(11,21)	(23, 2)	31
(8, 6)	(2, 8)	8
(27,20)	(11,10)	26
(27,20)	(2, 8)	37
(14, 3)	(2, 8)	17

Distance	Mouse	Cat
	108	74
Mouse evaded Cat		

Первая строка – название игры

Вторая строка – пустая

Третья строка – названия столбцов таблицы

Четвертая строка – строка разделительных символов

Пятая и следующие строки – позиции кота и мыши и расстояние между ними. Если на момент выполнения команды **P** позиция игрока неизвестна, то выводим (?, ?).

Под таблицей выводятся расстояния, пройденные обоими игроками, а также исход игры.

Файлы доступны по [ссылке](#).